

Reviewer Pack — Minimal Local Coherence of Groupoids and DPLL Proof Path Len...

Entry 455bff06297b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 06:37:17 UTC

Minimal Local Coherence of Groupoids and DPLL Proof Path Length Inequality

Entry ID: 455bff06297b **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 06:37:17 UTC

1. Conjecture

Field A (mathematical branch): Category Theory (Groupoids) **Field B** (complexity object): Boolean Satisfiability (DPLL Proof Complexity)

Statement:

For every CNF formula φ with n variables, the minimal local coherence of the groupoid associated with its clause structure is linearly correlated with its DPLL proof path length, such that $\text{LocalCoherence}(G(\varphi)) = \Theta(P(\varphi))$, where $P(\varphi)$ denotes the DPLL proof path length for φ .

Rationale (proposer's reasoning):

Local coherence in groupoids measures the complexity of interactions within a system, which could provide insights into the structural complexity of computational processes like DPLL proof searches. A higher local coherence implies more intricate interdependencies that may correspond to longer proof paths, suggesting a potential connection between categorical structures and algorithmic complexity.

Taxonomy category: GROUPOID_DPLL (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 36e965086b96e0d5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the computed correlation coefficient between $\text{LocalCoherence}(G(\varphi))$ and $P(\varphi)$ exceeds 0.7 for all CNF formulas φ with n variables ($n \leq 40$) across at least 25 of the 30 random seeds, with no seed producing a correlation coefficient below 0.5.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal local coherence" AND "groupoid" AND "DPLL proof path length" - "category theory" AND "Boolean satisfiability" AND "local coherence groupoid" - "CNF formula" AND "groupoid clause structure" AND "DPLL proof complexity"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2 * n):
            clause = [random.randint(-n, n) for _ in range(random.randint(1, 3))]
            clauses.append(clause)
        return clauses

    def dpll(cnf, assignment={}):
        if not cnf:
            return True
        literal = next((l for l in sorted(assignment.keys(), key=abs) if l not in assignment), None)
        if literal is None:
            return False

        def propagate(lit):
            new_cnf = []
            for clause in cnf:
                if lit in clause:
                    continue
                elif -lit in clause:
                    clause.remove(-lit)
                if not clause:
                    return None
            else:
                return new_cnf

        return dpll(propagate(literal), assignment | {literal: 1})
```

```

        new_cnf.append(clause)
    return new_cnf

pos, neg = propagate(lit), propagate(-lit)
if pos is None or neg is None:
    return False
return dpll(pos, assignment | {lit: True}) or dpll(neg, assignment | {-lit: True})

def local_coherence(cnf):
    n = len(cnf[0])
    graph = [[0] * n for _ in range(n)]
    for clause in cnf:
        for l1 in clause:
            for l2 in clause:
                if abs(l1) != abs(l2):
                    graph[abs(l1)-1][abs(l2)-1] += 1
    degree = [sum(row) for row in graph]
    return sum(degree) / (n * n)

def dpll_path_length(cnf):
    stack = [(cnf, {})]
    path_length = 0
    while stack:
        cnf, assignment = stack.pop()
        if not cnf:
            return path_length
        literal = next((l for l in sorted(assignment.keys(), key=abs) if l not in assignment), None)
        pos, neg = propagate(literal), propagate(-literal)
        if pos is None or neg is None:
            continue
        stack.append((pos, assignment | {literal: True}))
        stack.append((neg, assignment | {-literal: True}))
        path_length += 1
    return float('inf')

def propagate(lit):
    new_cnf = []
    for clause in cnf:
        if lit in clause:
            continue
        elif -lit in clause:
            clause.remove(-lit)
            if not clause:
                return None
        else:
            new_cnf.append(clause)
    return new_cnf

n = random.randint(5, 40)
cnf = generate_cnf(n)

local_coherence_value = local_coherence(cnf)
dpll_path_length_value = dpll_path_length(cnf)

return {
    "metric_name": "LocalCoherence vs DPLLPPathLength",

```

```

        "metric_value": local_coherence_value * dpll_path_length_value,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False if local_coherence_value == 0 or dpll_path_length_value == float('inf') else True,
        "counterexample": "" if local_coherence_value != 0 and dpll_path_length_value != float('inf') else "counterexample found"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
        support_fraction = 1.0
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        counterexample = "mapping_undefined"
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
        support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    print(f"RESULT: {'SUPPORTED' if support_fraction >= 0.8 else 'FALSIFIED'} mean={mean_value} std={std_value}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_de78bac2.py", line 114, in <module>
    trial_result = run_trial(seed)
                   ~~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_de78bac2.py", line 96, in run_trial
    local_coherence_value = local_coherence(cnf)
                           ~~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_de78bac2.py", line 60, in local_coherence
    graph[abs(l1)-1][abs(l2)-1] += 1
    ~~~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which means it was unable to compute the correlation coefficients as required by the support condition. | next: Re-run the test without errors to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14333
2	propose	ollama_remote	glm4:latest	0	0	12958
3	preregistration	ollama_remote	glm4:latest	0	0	9489
4	novelty	ollama_remote	glm4:latest	0	0	9205
5	novelty	ollama_remote	glm4:latest	0	0	8638
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14969
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15428
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8378
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13237
10	judge	ollama_remote	glm4:latest	0	0	26058

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 132694 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/455bff06297b.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/455bff06297b.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/455bff06297b.tar.gz (if generated)